DELIVERABLES

- **Complete Use Case diagram**
- **Use Case descriptions**

3.1 Refine Use Case Model

3.1.1 Finalize use case diagram with clear definition of actors and use cases

3.1.2 Keep refining user case descriptions. Describe clearly use case precondition, flow of events, and alternative flows.

3.1.3 Expect to re-visit your Use Case diagram and User Case descriptions to clarify and add details as you build the conceptual model and dynamic model

Documentation of Functional and Non-Functional Requirements

Functional Requirement

1. Users must be able to compare neighbourhoods (Where to Live).

1.1 Users must select neighbourhoods.

1.1.1 The system must display a list of available neighbourhoods, drawn from the in-scope URA Planning Areas defined in the Data Dictionary.

1.1.2 The user must select the neighbourhoods to compare.

1.1.2.1 The user must select at least 2 and at most 4 neighbourhoods.

1.1.2.2 Where the user attempts to proceed with fewer than 2 or more than 4 neighbourhoods selected, the system must block the comparison and display the allowed range.

1.2 Users may configure the comparison factors.

1.2.1 The system must display the six comparison factors defined in the Data Dictionary.

1.2.2 The user may include or exclude each comparison factor.

1.2.2.1 All comparison factors must be included by default.

1.2.2.2 The user must include at least one comparison factor.

1.2.2.3 The system must not request factor options for, calculate, or check data availability for an excluded comparison factor.

1.2.3 The user may specify the importance weight of each included comparison factor.

1.2.3.1 The importance weight must be expressed on a scale of 1 (least important) to 10 (essential).

1.2.3.2 Where the user does not specify an importance weight for an included comparison factor, the system must apply a default weight of 5.

1.2.4 The user may specify the preferred public transport mode where public transport is included.

1.2.4.1 The preferred mode must be one of MRT/LRT, Bus, or Either. The default must be Either.

1.2.5 The user must specify housing options where housing affordability is included.

1.2.5.1 The user must select one flat type: 3-room, 4-room, 5-room, or Executive.

1.2.5.2 The user must specify a budget range consisting of a minimum and a maximum resale price.

1.2.5.3 The minimum price must not exceed the maximum price.

1.2.5.4 The user may specify a minimum remaining lease of 60, 70, or 80 years.

1.2.6 The user may select the amenity types to be counted where amenities is included.

1.2.6.1 The amenity types must be supermarkets, hawker centres, shopping malls, libraries, gyms and sports facilities, and cafés.

1.2.6.2 All amenity types must be selected by default, and at least one amenity type must remain selected.

1.2.7 The user may select the healthcare facility type where healthcare is included.

1.2.7.1 The facility type must be one of GP clinics, polyclinics, or hospitals. The default must be GP clinics.

1.2.8 The user must specify commute details where commute is included.

1.2.8.1 The user must enter a 6-digit destination postal code.

1.2.8.2 The user must select a commute mode of public transport or drive. The default must be public transport.

1.2.8.3 The system must reject a destination postal code that cannot be resolved to a location in Singapore.

1.2.9 Greenery must have no factor options other than its importance weight.

1.3 Users must be able to view neighbourhood comparisons.

1.3.1 The system must retrieve the neighbourhood data for the selected neighbourhoods from the data snapshot.

1.3.2 The system must calculate a category score from 0 to 10 for each included comparison factor for each selected neighbourhood, using the user's factor options.

1.3.2.1 For public transport, amenities, greenery and healthcare, the category score must be derived from the neighbourhood's percentile rank among all in-scope neighbourhoods, as defined in the Data Dictionary.

1.3.2.2 For housing affordability, the category score must be the percentage of the neighbourhood's resale transactions in the last 12 months matching the user's flat type and minimum remaining lease whose price falls within the user's budget, divided by 10.

1.3.2.3 For commute, the category score must be 10 for a median travel time of 20 minutes or less, 0 for a median travel time of 80 minutes or more, and linearly interpolated between these values.

1.3.2.4 Accessibility measures must be calculated for each HDB block and aggregated to the neighbourhood as flat-weighted values.

1.3.3 Where data for an included comparison factor is unavailable for any selected neighbourhood, the system must exclude that factor for all selected neighbourhoods and must inform the user which factor was excluded and why.

1.3.3.1 Housing affordability data must be treated as unavailable where fewer than 10 matching resale transactions exist for a neighbourhood.

1.3.3.2 Where every included comparison factor is excluded under 1.3.3, the system must not produce a ranking and must ask the user to change the included factors.

1.3.4 The system must calculate an overall neighbourhood score for each selected neighbourhood as the weighted mean of its included category scores, as defined in the Data Dictionary.

1.3.5 The system must rank the selected neighbourhoods by overall neighbourhood score, rounded to 1 decimal place.

1.3.5.1 Where two or more neighbourhoods receive an equal overall neighbourhood score at 1 decimal place, the system must display them as joint-ranked rather than arbitrarily ordering them.

1.4 Users must receive a neighbourhood recommendation.

1.4.1 The system must identify the highest-ranked neighbourhood as the recommendation.

1.4.1.1 Where the highest rank is joint, the system must present all joint-ranked neighbourhoods as the recommendation.

1.4.2 The system must display the recommended neighbourhood with a one-sentence explanation naming the comparison factor with the largest marginal contribution over the runner-up.

1.4.2.1 The marginal contribution of a comparison factor must be calculated as (recommended neighbourhood's category score − runner-up's category score) × the user's importance weight.

1.4.3 Where the overall neighbourhood scores of the two highest-ranked neighbourhoods differ by 0.2 or less, the system must indicate that they are nearly tied.

1.4.4 The user may request a score breakdown.

1.4.4.1 The score breakdown must list, for each included comparison factor, the importance weight, the category scores and raw values of the recommended neighbourhood and the runner-up, and the marginal contribution, sorted by absolute marginal contribution.

1.4.4.2 The score breakdown must display the overall neighbourhood score formula with the user's importance weights.

1.4.4.3 The score breakdown must list each excluded comparison factor and state whether it was excluded by the user or because data was unavailable.

1.5 Users must be able to view comparison results visually.

1.5.1 The system must display the selected neighbourhoods as a ranked list showing each neighbourhood's overall neighbourhood score and category scores.

1.5.2 The system must display the raw value behind each category score (e.g. the percentage of flats within a 10-minute walk of an MRT/LRT exit).

1.5.3 The system must display the category scores as a grouped bar chart.

1.5.4 The system must display the boundaries of the selected neighbourhoods on a map, shaded by overall neighbourhood score, rendered from locally held Planning Area boundary geometry.

1.5.5 The system must display the generation date of the data snapshot used.

2. Users must be able to find events (Where to Go).

2.1 Users must specify event search criteria.

2.1.1 The system must display the event categories defined in the Data Dictionary.

2.1.2 The user must select at least 1 and at most 4 event categories.

2.1.3 The user must select a date option of today, this weekend, or a custom date range.

2.1.3.1 A custom date range must not start before the current date, and its end date must not be before its start date.

2.1.4 The user must provide a starting point by entering a postal code or by allowing the system to use the device location.

2.1.4.1 Where device location permission is denied, the system must request a postal code instead.

2.1.4.2 The system must reject a postal code that cannot be resolved to a location in Singapore.

2.1.5 The user must select a travel mode of public transport, drive, or walk.

2.1.6 The user must select a maximum travel time of 15, 30, 45, or 60 minutes.

2.2 The system must search for matching events.

2.2.1 The system must retrieve the stored events in the selected event categories whose dates overlap the selected date option.

2.2.2 The system must exclude events whose straight-line distance from the starting point cannot be covered within the maximum travel time.

2.2.3 Where more than 50 events remain after 2.2.2, the system must not calculate routes and must ask the user to narrow the event categories or dates.

2.2.4 The system must calculate the travel time and route from the starting point to each remaining event using the selected travel mode.

2.2.5 The system must exclude events whose travel time exceeds the maximum travel time.

2.2.6 Where a route cannot be calculated for an event, the system must display the straight-line distance labelled as estimated instead of a travel time.

2.3 Users must be able to view event results.

2.3.1 The system must display event results in pages of 10.

2.3.2 Each event result must show the title, event category, date and time, venue, travel time, route summary, and a link to the source post.

2.3.3 Where no events match the search criteria, the system must display a message suggesting a wider date range or a longer maximum travel time.

2.3.4 The system must display how long ago events were last updated.

2.3.4.1 Where events have not been updated for more than 24 hours, the system must display a warning.

2.4 Users may sort and filter event results.

2.4.1 The user may sort event results by shortest travel time, soonest, or recently added. The default must be shortest travel time.

2.4.2 The user may hide or show the results of any selected event category.

2.4.3 Sorting and hiding results must not trigger a new search or any external data request.

2.5 Users may view the route to an event.

2.5.1 The user may select an event result to view its route.

2.5.2 The system must display a map showing the starting point, the venue, and the route, together with step-by-step directions.

2.5.3 When the user returns to the results, the system must restore the previous sort order, hidden categories, and page.

2.6 Users travelling by car must be shown parking availability.

2.6.1 Where the travel mode is drive, the system must display for each event the carparks within 500 m of the venue and their number of available lots.

2.6.2 Displayed parking availability must be no older than 2 minutes.

2.6.3 Where no carpark lies within 500 m of a venue, the system must display a message stating that no nearby carparks were found.

2.6.4 Where parking availability cannot be retrieved, the system must display that parking information is unavailable and must continue to display the event results.

3. The system must maintain neighbourhood and event data.

3.1 The system must maintain the data snapshot.

3.1.1 The system must provide a data refresh procedure, run by the Maintainer, that regenerates the data snapshot from the Data Sources.

3.1.2 The data snapshot must be refreshed at least every 3 months.

3.1.3 The system must validate a new data snapshot before it replaces the active data snapshot.

3.1.4 Where a refresh or its validation fails, the system must continue operating using the most recent successful data snapshot.

3.1.5 Each data snapshot must record its generation date and the versions of the source datasets used.

3.1.6 Where the active data snapshot is older than 12 months, the system must display a warning with neighbourhood comparison results.

3.2 The system must ingest events.

3.2.1 The system must collect new posts every hour from the Telegram Channel List.

3.2.2 The system must extract event details from each post using the LLM Service, in the Event format defined in the Data Dictionary.

3.2.3 The system must discard any extracted event that has no title, start date-time, or location, whose event category is not defined in the Data Dictionary, or that has already ended.

3.2.4 The system must discard any event whose location cannot be resolved to coordinates in Singapore.

3.2.5 The system must merge duplicate events, defined as events on the same date with similar titles and locations within 200 m, retaining all source links.

3.2.6 The system must store only the extracted event details and the link to the source post, and must not store raw message text or the identity of the poster.

3.2.7 Where a Telegram channel or the LLM Service is unavailable, the system must retry at the next run without losing unprocessed posts.

3.2.8 The system must remove events that have ended at least once per day.

Non-Functional Requirement

Category	Requirement
Usability	At least 4 out of 5 first-time users in a moderated usability test shall be able to complete a full neighbourhood comparison (selecting neighbourhoods, configuring factors, and viewing a recommendation) within 10 minutes, and a full event search within 5 minutes, without external help.
Performance	The system shall display neighbourhood comparison results within 3 seconds of the user submitting, or within 10 seconds where the commute factor is included. The system shall display event search results within 8 seconds of the user submitting, and re-sorted or filtered event results within 1 second.
Reliability	The system shall handle invalid user input and the unavailability of any external service (OneMap API, LTA DataMall, Telegram, LLM Service, Data Sources) without crashing or terminating unexpectedly, and shall degrade as specified in the functional requirements.

Supportability	The system shall be built from modular components, such that a new comparison factor, event category, or data source can be added without modifying the core scoring engine, and the Telegram Channel List can be changed through configuration without code changes.
Scalability	The system should continue to meet the Performance targets above as the number of in-scope neighbourhoods, stored events, and Telegram channels grows.
Accuracy	Given the same data snapshot and user inputs, a manual recalculation using the documented scoring formulas shall equal the system's category scores and overall neighbourhood scores to 1 decimal place (±0.05).
Transparency	The system shall display enough information about how a recommendation was calculated (per FR 1.4.2–1.4.4 and FR 1.5.2) that a user can identify which comparison factors drove the result. Every event result shall link to its source post so that the user can verify the extracted details.
Data Currency	Under normal operation, events posted to the Telegram Channel List shall be available to users within 1 hour of posting, displayed parking availability shall be no older than 2 minutes, and the data snapshot shall be no older than 3 months.
Compatibility	The system shall operate correctly on the current stable versions of Google Chrome and Mozilla Firefox on desktop Windows and macOS.
Security	The system shall not require account creation and shall not collect personally identifiable information. Importance weights, factor options, postal codes, and device location shall be held only for the current session and shall not be stored or logged. API credentials shall be held on the server and never sent to the browser. Raw Telegram message text and poster identities shall never be stored.

Data Dictionary

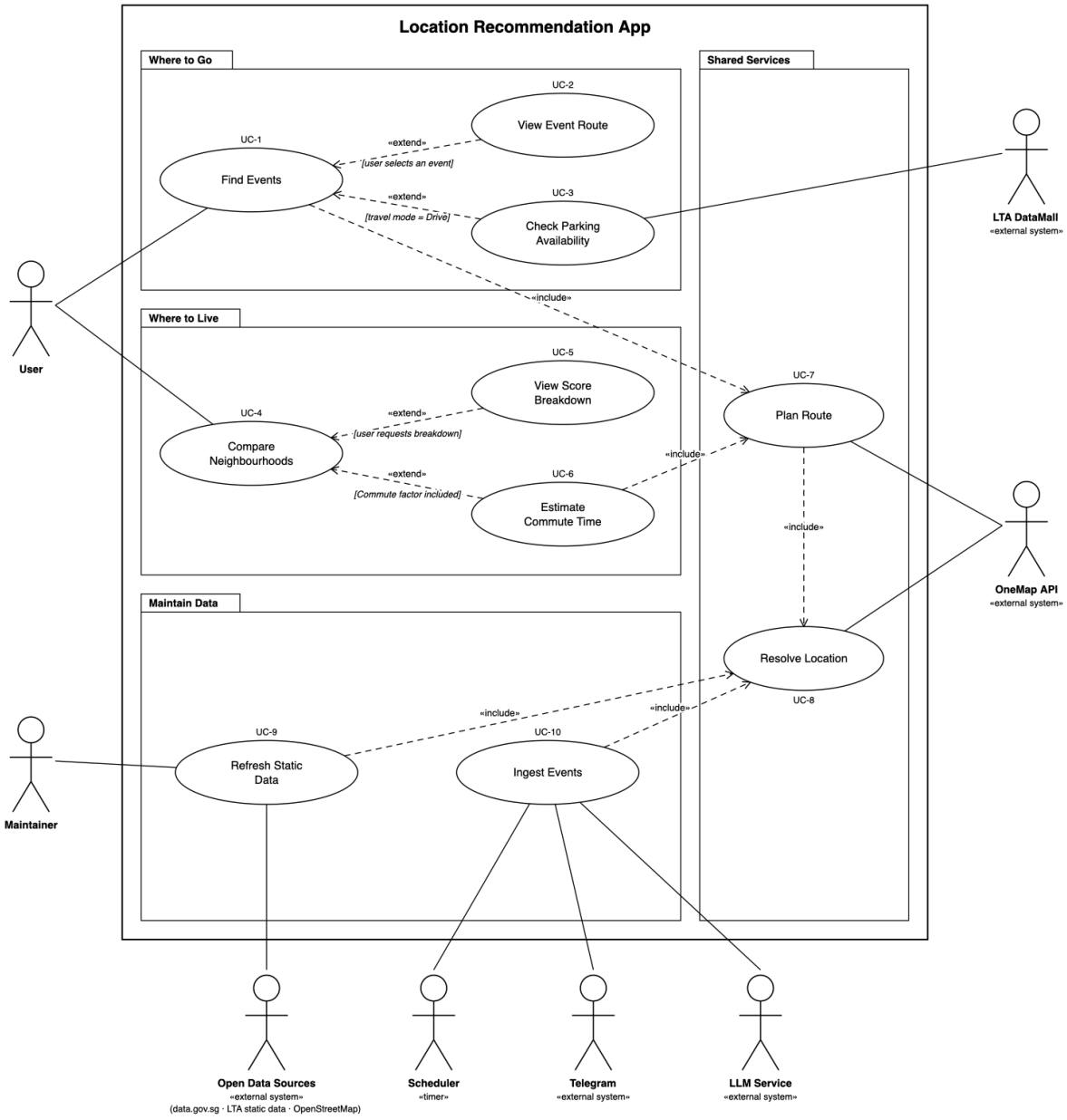
Term	Definition
Neighbourhood	A URA Planning Area that contains at least one HDB residential block (an in-scope Planning Area), used by the system as the geographical unit for selecting and comparing locations in Where to Live.
Where to Live	The feature that compares 2 to 4 neighbourhoods against the user's included comparison factors, importance weights, and factor options, and recommends one.
Where to Go	The feature that finds upcoming events matching the user's event categories, date option, and travel limits, and shows how to travel to them.
Comparison Factor	One of the six criteria the system uses to evaluate neighbourhoods: public transport, housing affordability, amenities, greenery, healthcare, and commute.
Included Factor	A comparison factor the user has chosen to use in a comparison. An excluded factor is not calculated and does not affect any score.
Importance Weight	A value from 1 (least important) to 10 (essential) that the user assigns to an included factor, representing how much it influences the Overall Neighbourhood Score. The default is 5.
Factor Options	The user's choices that change how an included factor is measured: preferred public transport mode; flat type, budget range, and minimum remaining lease; amenity types; healthcare facility type; and commute destination and mode.
Public Transport	Walking access from HDB blocks to the user's preferred mode (MRT/LRT exit, Qualifying Bus Stop, or the nearer of the two). Category Score = 0.7 × Percentile Rank of the flat-weighted percentage of flats within a 10-minute walk + 0.3 × Percentile Rank of the flat-weighted median Walking Time (shorter is better).
Qualifying Bus Stop	A bus stop served by at least 3 bus services.

Housing Affordability	The percentage of a neighbourhood's HDB resale transactions in the last 12 months, matching the user's flat type and minimum remaining lease, whose price falls within the user's budget range. Category Score = percentage ÷ 10.
Amenities	The flat-weighted median number of the user's selected amenity types within 800 m of each HDB block, with each amenity type counted up to a maximum of 3. Category Score = Percentile Rank.
Greenery	The flat-weighted percentage of flats within 400 m of a park or park connector. Category Score = Percentile Rank.
Healthcare	The flat-weighted median distance from HDB blocks to the nearest facility of the user's selected type. Category Score = Percentile Rank (shorter is better).
Commute	The median travel time from 5 sample HDB blocks in a neighbourhood to the user's destination by the selected commute mode. Category Score = 10 at 20 minutes or less, 0 at 80 minutes or more, linearly interpolated between.
HDB Block	A residential block listed in HDB Property Information, with its resolved coordinates and number of flats.
Flat-Weighted	Aggregated across a neighbourhood's HDB blocks with each block counted in proportion to its number of flats.
Walking Time	Straight-line distance × 1.3 (detour factor) ÷ 80 metres per minute.
Neighbourhood Data	The per-block and per-neighbourhood measures held in the data snapshot for public transport, housing affordability, amenities, greenery, and healthcare.
Data Snapshot	The validated set of Neighbourhood Data produced by the data refresh procedure, recording its generation date and source dataset versions. Where to Live reads only from the active data snapshot at runtime.
Category Score	A value from 0 to 10 representing how well a neighbourhood performs on a single included factor, before importance weights are applied.

Percentile Rank	A neighbourhood's position among all in-scope neighbourhoods on a measure, scaled to 0–10: (rank − 1) ÷ (number of in-scope neighbourhoods − 1) × 10, where rank 1 is the worst-performing neighbourhood.
Overall Neighbourhood Score	Σ (Category Score × Importance Weight) ÷ Σ (Importance Weight), over the included factors not excluded for unavailable data. A value from 0 to 10, rounded to 1 decimal place for ranking.
Joint-Ranked	The ranking given to two or more neighbourhoods whose Overall Neighbourhood Scores are equal at 1 decimal place.
Marginal Contribution	(Recommended neighbourhood's Category Score − runner-up's Category Score) × Importance Weight. A positive value favours the recommended neighbourhood; a negative value favours the runner-up.
Recommendation	The highest-ranked selected neighbourhood (or all joint-ranked neighbourhoods at the highest rank), together with the comparison factor with the largest Marginal Contribution.
Score Breakdown	A detailed view listing each included factor's Importance Weight, Category Scores, raw values, and Marginal Contribution, the overall score formula, and all excluded factors.
Event	An upcoming activity extracted from a Telegram post, consisting of: title, Event Category, start date-time, end date-time (optional), venue name, address, summary, resolved coordinates, Planning Area, and link(s) to the source post(s).
Event Category	One of: Food & Markets, Arts & Culture, Music & Performances, Sports & Fitness, Family & Kids, Workshops & Classes, Community, and Sales & Pop-ups.
Starting Point	The location from which travel to events is calculated, given as a postal code or the device location, and held for the current session only.

Travel Mode	The means of travel used for routing: public transport, drive, or walk in Where to Go; public transport or drive for commute.
Maximum Travel Time	The longest travel time, of 15, 30, 45, or 60 minutes, that the user accepts for an event.
Route	The travel time, distance, route summary, and step-by-step directions between two locations for a Travel Mode, obtained from OneMap API.
Parking Availability	The number of available lots at carparks within 500 m of an event venue, obtained from LTA DataMall.
Telegram Channel List	The list of public Telegram broadcast channels, maintained by the project team in configuration, from which events are collected.
Maintainer	A project team member who runs the data refresh procedure and maintains the Telegram Channel List.
Data Sources	data.gov.sg and government agency datasets (URA, HDB, LTA, NParks, MOH, NEA, SFA, NLB) and OpenStreetMap, from which the data snapshot is generated.
OneMap API	The Singapore Land Authority's mapping service, used to resolve locations, calculate routes, and provide base map tiles.
LTA DataMall	The Land Transport Authority's data service, used for bus stop data and live carpark availability.
LLM Service	A large language model API used to extract Event details from Telegram post text.
Session	The period for which the user's browser tab remains open. Inputs held for the session are discarded when it ends.

Use Case Diagram



Use Case Description

Use Case ID:	UC-1
Use Case Name:	Find Events

Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	User (initiating)
Description:	The User searches for upcoming events matching their chosen event categories, date option, and travel limits, and views the matching events with the travel time from their starting point.
Preconditions:	At least one successful event ingestion run has completed (UC-10), so the event store is available.
Postconditions:	Matching events are displayed with their travel time and route summary, sorted in the User's chosen order. The User's starting point is held for the current session only and is not stored.
Priority:	High
Frequency of Use:	Once or more per session; the User may revise the search at any time.

Flow of Events:	1. User opens Where to Go. 2. System displays the event categories defined in the Data Dictionary and the search criteria (FR 2.1.1). 3. User selects between 1 and 4 event categories (FR 2.1.2). 4. User selects a date option: today, this weekend, or a custom date range (FR 2.1.3). 5. User enters a starting postal code or chooses to use the device location (FR 2.1.4). 6. User selects a travel mode and a maximum travel time, and submits the search (FR 2.1.5, FR 2.1.6). 7. System retrieves the stored events in the selected categories whose dates overlap the selected date option (FR 2.2.1). 8. System excludes events whose straight-line distance from the starting point cannot be covered within the maximum travel time (FR 2.2.2). 9. System invokes UC-7 to calculate the travel time and route to each remaining event, and excludes events whose travel time exceeds the maximum travel time (FR 2.2.4, FR 2.2.5). 10. System displays the results in pages of 10, sorted by shortest travel time, each showing the title, event category, date and time, venue, travel time, route summary, and a link to the source post (FR 2.3.1, FR 2.3.2, FR 2.4.1). 11. System displays how long ago events were last updated (FR 2.3.4). 12. User may change the sort order or hide/show an event category; the system reorders or filters the displayed results without a new search (FR 2.4.1–FR 2.4.3).
Alternative Flows:	A1 — User chooses the device location and denies location permission: system requests a postal code instead (FR 2.1.4.1); returns to step 5. A2 — More than 50 events remain after step 8: system asks the User to narrow the event categories or dates without calculating routes (FR 2.2.3); returns to step 3. A3 — No events remain after step 9: system displays a message suggesting a wider date range or a longer maximum travel time (FR 2.3.3); the use case ends.

	A4 — Events have not been updated for more than 24 hours: system displays a warning with the results (FR 2.3.4.1); continues at step 12.
Exceptions:	E1 — The starting postal code cannot be resolved to a location in Singapore: system displays that the postal code is not recognised (FR 2.1.4.2); returns to step 5. E2 — UC-7 cannot calculate a route for an event: system displays the straight-line distance labelled as estimated for that event, and lists it after routed events when sorting by shortest travel time (FR 2.2.6); continues at step 10. E3 — The event store is unavailable: system displays an error message and the use case terminates.
Includes:	UC-7 Plan Route
Special Requirements:	Results must be displayed within 8 seconds of submission, and re-sorted or filtered results within 1 second (Performance NFR). Sorting and hiding categories must not trigger any external data request (FR 2.4.3). The starting point must not be stored beyond the session (Security NFR).
Assumptions:	Event details are extracted automatically from Telegram posts and may contain errors, so each result links to its source post for the User to verify.
Notes and Issues:	Extended by UC-2 at extension point "event selected" (step 12), under the guard [user selects an event]. Extended by UC-3 at extension point "travel mode is drive" (step 10), under the guard [travel mode = Drive]. The 50-event limit in A2 caps the number of UC-7 invocations per search at 50.

Use Case ID:	UC-2		
Use Case Name:	View Event Route		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	User (initiating)
Description:	The User selects an event from the results and views the full route from their starting point to the venue on a map.
Preconditions:	UC-1 has displayed at least one event result.
Postconditions:	The route to the selected event has been displayed. No stored state changes.
Priority:	Medium
Frequency of Use:	Optional; 0 to n times per search, at the User's discretion.
Flow of Events:	1. User selects an event result (FR 2.5.1). 2. System retrieves the route calculated for that event during UC-1. 3. System displays a map showing the starting point, the venue, and the route, together with step-by-step directions (FR 2.5.2). 4. User returns to the results; system restores the previous sort order, hidden categories, and page (FR 2.5.3).
Alternative Flows:	A1 — The travel mode is walk or drive: the directions contain road and path steps only, with no public transport legs; continues at step 3.
Exceptions:	E1 — No route was calculated for the selected event (UC-1 E2): system displays the venue on the map with the estimated straight-line distance and states that directions are unavailable; continues at step 4.
Includes:	None.
Special Requirements:	The map must be rendered with OneMap API base map tiles.
Assumptions:	Directions reflect OneMap's scheduled public transport services at the time of the search, not real-time conditions.

Notes and Issues:	«extend» of UC-1 at extension point "event selected"; guard condition [user selects an event]. Reuses the route calculated in UC-1 rather than invoking UC-7 again, so it makes no additional external requests.

Use Case ID:	UC-3		
Use Case Name:	Check Parking Availability		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Initiating actor: NULL (triggered within UC-1). Participating actor: LTA DataMall.
Description:	When the User travels by car, the system shows live carpark availability near each event venue.
Preconditions:	UC-1 is in progress and the selected travel mode is drive.
Postconditions:	Each displayed event shows the carparks within 500 m of its venue and their available lots, or a message stating that parking information is unavailable.
Priority:	Medium
Frequency of Use:	Conditional; once per event search in drive mode.
Flow of Events:	1. System requests current carpark availability from LTA DataMall. 2. System identifies the carparks within 500 m of each event venue (FR 2.6.1). 3. System displays, with each event, the carpark name, number of available lots, distance from the venue, and time of last update (FR 2.6.1). 4. Control returns to UC-1.

Alternative Flows:	A1 — Carpark availability was retrieved less than 2 minutes ago: system reuses the retrieved data instead of requesting it again (FR 2.6.2); continues at step 2. A2 — No carpark lies within 500 m of a venue: system displays that no nearby carparks were found for that event (FR 2.6.3); continues at step 4.
Exceptions:	E1 — LTA DataMall is unavailable: system displays that parking information is currently unavailable, and UC-1 continues to display the event results (FR 2.6.4).
Includes:	None.
Special Requirements:	Displayed parking availability must be no older than 2 minutes (FR 2.6.2, Data Currency NFR).
Assumptions:	LTA DataMall covers HDB, URA, and major LTA-listed carparks; private carparks outside the feed are not shown. Availability may change before the User arrives.
Notes and Issues:	«extend» of UC-1 at extension point "travel mode is drive"; guard condition [travel mode = Drive]. Not triggered directly by the User, which is why the initiating actor is NULL.

Use Case ID:	UC-4		
Use Case Name:	Compare Neighbourhoods		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	User (initiating)
Description:	The User selects 2 to 4 neighbourhoods, chooses which comparison factors to include with their importance weights and factor options, and receives a ranking, a recommended neighbourhood, and an explanation.

Preconditions:	A validated data snapshot exists (UC-9).
Postconditions:	The selected neighbourhoods are displayed as a ranked list with category scores, raw values, a chart, and a map, and the recommendation is displayed with its explanation. All inputs are held for the current session only.
Priority:	High
Frequency of Use:	Once or more per session; re-runs whenever the selection or inputs change.
Flow of Events:	1. User opens Where to Live. 2. System displays the list of in-scope neighbourhoods (FR 1.1.1). 3. User selects between 2 and 4 neighbourhoods (FR 1.1.2). 4. System displays the six comparison factors, all included with a default importance weight of 5 (FR 1.2.1, FR 1.2.2.1, FR 1.2.3.2). 5. User may exclude comparison factors and, for each included factor, may set its importance weight from 1 to 10 and its factor options (FR 1.2.2–FR 1.2.9). 6. User submits the comparison. 7. System validates the selection and inputs (FR 1.1.2.1, FR 1.2.2.2, FR 1.2.5.3, FR 1.2.8.1). 8. System retrieves the neighbourhood data for the selected neighbourhoods from the data snapshot (FR 1.3.1). 9. System calculates a category score for each included factor for each selected neighbourhood, using the User's factor options (FR 1.3.2). 10. System calculates each neighbourhood's overall neighbourhood score as the weighted mean of its included category scores (FR 1.3.4). 11. System ranks the selected neighbourhoods by overall neighbourhood score (FR 1.3.5). 12. System displays the ranked list with each category score and its raw value, the grouped bar chart, and the map (FR 1.5.1–FR 1.5.4).

	13. System displays the recommended neighbourhood with a one-sentence explanation naming the factor with the largest marginal contribution over the runner-up (FR 1.4.1, FR 1.4.2). 14. System displays the generation date of the data snapshot (FR 1.5.5).
Alternative Flows:	A1 — Two or more neighbourhoods have equal overall neighbourhood scores at 1 decimal place: system displays them as joint-ranked (FR 1.3.5.1); continues at step 12. A2 — The highest rank is joint: system presents all joint-ranked neighbourhoods as the recommendation (FR 1.4.1.1); continues at step 14. A3 — The two highest overall neighbourhood scores differ by 0.2 or less: system indicates that they are nearly tied (FR 1.4.3); continues at step 14. A4 — The data snapshot is older than 12 months: system displays a warning with the results (FR 3.1.6); the use case ends. A5 — User changes the selection or inputs after results have been displayed: returns to step 7.
Exceptions:	E1 — Fewer than 2 or more than 4 neighbourhoods are selected: system blocks the comparison and displays the allowed range (FR 1.1.2.2); returns to step 3. E2 — No factor is included, a required factor option is missing, or the budget minimum exceeds the maximum: system identifies the invalid input (FR 1.2.2.2, FR 1.2.5, FR 1.2.8); returns to step 5. E3 — The commute destination postal code cannot be resolved to a location in Singapore: system displays that the postal code is not recognised (FR 1.2.8.3); returns to step 5. E4 — Data for an included factor is unavailable for any selected neighbourhood: system excludes that factor for all selected neighbourhoods and informs the User which factor was excluded and why (FR 1.3.3); continues at step 10. E5 — Every included factor is excluded under E4: system displays that the neighbourhoods cannot be compared with the chosen factors (FR 1.3.3.2); returns to step 5. E6 — No validated data snapshot exists: system displays an error message and the use case terminates.

Includes:	None.
Special Requirements:	Results must be displayed within 3 seconds of submission, or within 10 seconds where commute is included (Performance NFR). A manual recalculation using the documented scoring formulas must equal the displayed scores to 1 decimal place (Accuracy NFR).
Assumptions:	A neighbourhood is an in-scope URA Planning Area. Accessibility measures cover HDB blocks only and are flat-weighted; private housing is not represented. Preferences are compensatory, so a low category score can be offset by a high one and a weighted mean is a defensible model.
Notes and Issues:	Extended by UC-6 at extension point "commute included" (step 9), under the guard [Commute factor included]. Extended by UC-5 at extension point "breakdown requested" (step 13), under the guard [user requests breakdown]. Selecting neighbourhoods, configuring factors, scoring, and displaying results are steps of this single goal-level use case rather than separate use cases, as none of them delivers value to the User on its own.

Use Case ID:	UC-5		
Use Case Name:	View Score Breakdown		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	User (initiating)
Description:	The User opens a detailed breakdown showing how each comparison factor contributed to the recommendation relative to the runner-up.
Preconditions:	UC-4 has displayed a ranking and a recommendation.

Postconditions:	The contribution of each included factor has been displayed. No stored state changes.
Priority:	Medium
Frequency of Use:	Optional; 0 to n times per comparison, at the User's discretion.
Flow of Events:	1. User requests the score breakdown (FR 1.4.4). 2. System calculates the marginal contribution of each included factor as (recommended neighbourhood's category score − runner-up's category score) × the User's importance weight (FR 1.4.2.1). 3. System displays each included factor's importance weight, both neighbourhoods' category scores and raw values, and its marginal contribution, sorted by absolute marginal contribution (FR 1.4.4.1). 4. System displays the overall neighbourhood score formula with the User's importance weights (FR 1.4.4.2). 5. System lists each excluded factor and whether it was excluded by the User or because data was unavailable (FR 1.4.4.3). 6. Control returns to the results displayed by UC-4.
Alternative Flows:	A1 — The highest rank is joint: system compares the joint-ranked neighbourhoods with each other and states which factors each scores higher on; continues at step 4.
Exceptions:	None identified. Every path returns control to UC-4.
Includes:	None.
Special Requirements:	The breakdown must contain enough information for a User to identify which comparison factors drove the result (Transparency NFR). Positive marginal contributions favour the recommended neighbourhood; negative marginal contributions favour the runner-up.
Assumptions:	Users read the breakdown to sanity-check a recommendation, not to audit the source datasets.

Notes and Issues:	«extend» of UC-4 at extension point "breakdown requested"; guard condition [user requests breakdown]. Marginal contribution is used instead of category score × importance weight, because the latter mainly reflects the User's own highest weight rather than why one neighbourhood outscored another.

Use Case ID:	UC-6		
Use Case Name:	Estimate Commute Time		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Initiating actor: NULL (triggered within UC-4).
Description:	When the commute factor is included, the system estimates the typical travel time from each selected neighbourhood to the User's destination and converts it into a commute category score.
Preconditions:	UC-4 is in progress, commute is included, and a destination postal code and commute mode have been provided.
Postconditions:	Each selected neighbourhood holds a median commute time and a commute category score, returned to UC-4.
Priority:	High
Frequency of Use:	Conditional; once per comparison with commute included.

Flow of Events:	1. System selects 5 sample HDB blocks spread across each selected neighbourhood. 2. System invokes UC-7 to calculate the travel time from each sample block to the destination using the selected commute mode. 3. System takes the median travel time of the samples as the neighbourhood's commute time. 4. System converts the commute time into a category score (FR 1.3.2.3). 5. Control returns to UC-4 at step 10.
Alternative Flows:	A1 — The commute time for the same neighbourhood, destination, and commute mode was already calculated in this session: system reuses it; continues at step 4.
Exceptions:	E1 — UC-7 fails for some sample blocks but succeeds for at least 3: system uses the median of the successful samples; continues at step 4. E2 — UC-7 succeeds for fewer than 3 sample blocks of a neighbourhood: commute data is treated as unavailable (UC-4 E4, FR 1.3.3). E3 — The destination postal code cannot be resolved (UC-7 E1): control returns to UC-4 E3.
Includes:	UC-7 Plan Route
Special Requirements:	At most 20 route calculations per comparison (4 neighbourhoods × 5 sample blocks), requested in parallel. The destination postal code must not be stored beyond the session (Security NFR).
Assumptions:	The median of 5 sample blocks represents the typical commute time for a neighbourhood. Travel times are OneMap estimates rather than live traffic conditions.
Notes and Issues:	«extend» of UC-4 at extension point "commute included"; guard condition [Commute factor included]. Commute depends on the User's destination, so it cannot be precomputed in the data snapshot and is calculated at request time. This is why it uses a fixed scale instead of percentile rank.

Use Case ID:	UC-7		
Use Case Name:	Plan Route		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Initiating actor: NULL (invoked by UC-1 or UC-6). Participating actor: OneMap API.
Description:	Calculates the travel time, distance, route summary, and step-by-step directions between two locations for a given travel mode.
Preconditions:	The invoking use case has provided an origin, a destination, and a travel mode.
Postconditions:	The travel time, distance, route summary, and directions are returned to the invoking use case, or a failure is returned.
Priority:	High
Frequency of Use:	Up to 50 times per event search (UC-1) and up to 20 times per comparison with commute included (UC-6).
Flow of Events:	1. System receives the origin, destination, and travel mode from the invoking use case. 2. System invokes UC-8 to resolve any origin or destination not already held as validated coordinates. 3. System requests a route for the travel mode from OneMap API. 4. System extracts the total travel time, distance, a route summary (e.g. "Bus 69 → East-West Line"), and step-by-step directions. 5. System returns the result to the invoking use case.
Alternative Flows:	A1 — The OneMap API access token has expired: system obtains a new token and repeats step 3 once.

Exceptions:	E1 — UC-8 cannot resolve a location: system returns a failure to the invoking use case. E2 — OneMap API is unavailable, does not respond within 5 seconds, or finds no route: system returns a failure, and the invoking use case applies its own fallback (UC-1 E2, UC-6 E1–E2).
Includes:	UC-8 Resolve Location
Special Requirements:	Each route request must time out after 5 seconds. The OneMap API access token must be held on the server and never sent to the browser (Security NFR).
Assumptions:	OneMap public transport routing reflects scheduled services, not real-time delays.
Notes and Issues:	«include» of UC-1 and UC-6 — it always executes when either of them calculates travel time, which is why it carries no guard condition. Shared by both features, which is the reuse the include relationship represents.

Use Case ID:	UC-8		
Use Case Name:	Resolve Location		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Initiating actor: NULL (invoked by UC-7, UC-9, or UC-10). Participating actor: OneMap API.
Description:	Converts a location input (a postal code, device coordinates, or address text) into validated coordinates in Singapore and identifies the Planning Area containing them.
Preconditions:	The invoking use case has provided a location input and its type.

Postconditions:	Validated coordinates and the containing Planning Area are returned to the invoking use case, or a failure is returned.
Priority:	High
Frequency of Use:	Very frequent; for every search, route calculation, ingested event, and new HDB block address.
Flow of Events:	1. System receives the location input and its type. 2. Where the input is a postal code, system checks that it has 6 digits. 3. Where the input is a postal code or address text, system searches OneMap API and takes the best match. 4. System checks that the coordinates lie within Singapore. 5. System identifies the Planning Area containing the coordinates, using the boundary geometry in the data snapshot. 6. System returns the coordinates and Planning Area to the invoking use case.
Alternative Flows:	A1 — The input is device coordinates: system skips steps 2–3; continues at step 4. A2 — The same input was resolved earlier: system reuses the stored result; continues at step 6. A3 — The coordinates lie outside every in-scope Planning Area (e.g. an offshore venue): system returns the coordinates with no Planning Area, which remains valid for routing; continues at step 6.
Exceptions:	E1 — The postal code does not have 6 digits: system returns the failure "invalid format". E2 — OneMap API finds no match: system returns the failure "not found". E3 — The coordinates lie outside Singapore: system returns the failure "outside Singapore". E4 — OneMap API is unavailable: system returns the failure "service unavailable".
Includes:	None.
Special Requirements:	Resolved results must be cached so that repeated inputs do not trigger repeated OneMap API requests.

Assumptions:	OneMap's best match for a venue name is usually correct; vague venue names may resolve to the wrong place.
Notes and Issues:	«include» of UC-7, UC-9, and UC-10. The invoking use case decides how a failure is presented to the User or Maintainer.

Use Case ID:	UC-9		
Use Case Name:	Refresh Static Data		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Maintainer (initiating). Participating actor: Data Sources.
Description:	The Maintainer runs the data refresh procedure, which downloads the static datasets, calculates per-block measures, and replaces the data snapshot used by Where to Live.
Preconditions:	The Maintainer has access to the data refresh procedure and valid credentials for the Data Sources.
Postconditions:	A new validated data snapshot, recording its generation date and source dataset versions, is active; or the previous data snapshot remains active if the refresh failed.
Priority:	High
Frequency of Use:	At least every 3 months (FR 3.1.2), and whenever a source dataset is updated.

Flow of Events:	1. Maintainer runs the data refresh procedure (FR 3.1.1). 2. System downloads the Planning Area boundaries, HDB Property Information, HDB resale transactions, MRT/LRT exits, bus stops and services, parks and park connectors, healthcare facilities, and amenity locations from the Data Sources. 3. System invokes UC-8 to resolve the coordinates of each HDB block. 4. System identifies the in-scope neighbourhoods defined in the Data Dictionary. 5. System calculates for each HDB block the walking time to the nearest MRT/LRT exit and Qualifying Bus Stop, the distance to the nearest park or park connector and to the nearest facility of each healthcare type, and the number of each amenity type within 800 m. 6. System assigns each resale transaction to its HDB block and neighbourhood. 7. System writes a new data snapshot recording its generation date and source dataset versions (FR 3.1.5). 8. System validates the new data snapshot (FR 3.1.3). 9. System replaces the active data snapshot with the new data snapshot. 10. System reports a summary of neighbourhoods, blocks, transactions, and warnings to the Maintainer.
Alternative Flows:	A1 — An HDB block's address is unchanged from the previous data snapshot: system reuses its stored coordinates instead of invoking UC-8 for that block; continues at step 4.
Exceptions:	E1 — A dataset cannot be downloaded: system stops the refresh, keeps the previous data snapshot active, and reports the dataset that failed (FR 3.1.4). E2 — An HDB block address cannot be resolved: system excludes that block and lists it in the warnings; continues at step 4. E3 — The new data snapshot fails validation: system discards it, keeps the previous data snapshot active, and reports the reasons (FR 3.1.4).

	E4 — The refresh fails and no previous data snapshot exists: Where to Live remains unavailable until a refresh succeeds (UC-4 E6).
Includes:	UC-8 Resolve Location
Special Requirements:	The active data snapshot must only be replaced after the new data snapshot passes validation. The web application must not depend on the Data Sources being reachable at runtime.
Assumptions:	Static datasets change slowly, so a data snapshot up to 3 months old supports a defensible comparison. Walking time is estimated as straight-line distance × 1.3 ÷ 80 m per minute.
Notes and Issues:	Replaces the earlier "Refresh Government Neighbourhood Data" and "Serve Last-Known-Good Snapshot" use cases. The refresh is now run by the Maintainer, so it has an initiating actor. Keeping the last successful snapshot on failure is handled as exceptions E1 and E3 of this use case rather than as a separate use case, as it delivers no separate goal.

Use Case ID:	UC-10		
Use Case Name:	Ingest Events		
Created By:	SC2006 Team 5	**Last Updated By:**	SC2006 Team 5
Date Created:	17 Sep 2026	**Date Last Updated:**	17 Sep 2026

Actor:	Initiating actor: NULL (system-triggered on an hourly schedule). Participating actors: Telegram, LLM Service.
Description:	Every hour, the system collects new posts from the Telegram Channel List, extracts structured event details with the LLM Service, validates and locates each event, and stores it for Where to Go.
Preconditions:	The Telegram Channel List exists in configuration, and credentials for Telegram and the LLM Service are configured.

Postconditions:	New valid events are stored with their source links, ended events are removed, and the ingestion time is recorded.
Priority:	High
Frequency of Use:	Hourly (FR 3.2.1).
Flow of Events:	1. System starts the ingestion run on its hourly schedule (FR 3.2.1). 2. System retrieves, for each channel in the Telegram Channel List, the posts published since the last processed post. 3. System sends each post's text to the LLM Service to extract event details in the Event format defined in the Data Dictionary (FR 3.2.2). 4. LLM Service returns zero or more events for the post. 5. System discards events that fail validation (FR 3.2.3). 6. System invokes UC-8 to resolve each remaining event's location. 7. System merges duplicate events, retaining all source links (FR 3.2.5). 8. System stores each event's details and source link (FR 3.2.6). 9. System records the last processed post for each channel and the ingestion time. 10. On the first run of each day, system removes events that have ended (FR 3.2.8).
Alternative Flows:	A1 — LLM Service returns no events because the post is not about an event: system skips the post; continues at step 3 with the next post. A2 — The previous ingestion run is still in progress: system skips this run and the use case ends.

Exceptions:	E1 — A channel cannot be reached: system skips that channel for this run and logs the failure; it is retried at the next run (FR 3.2.7). E2 — LLM Service is unavailable: system stops processing further posts without advancing the last processed post, so the remaining posts are retried at the next run (FR 3.2.7). E3 — LLM Service output does not match the Event format: system discards the output for that post and logs it; continues at step 3 with the next post. E4 — An event's location cannot be resolved (UC-8 fails): system discards the event (FR 3.2.4); continues at step 7.
Includes:	UC-8 Resolve Location
Special Requirements:	Only extracted event details and the source link may be stored; raw message text and poster identities must never be stored (FR 3.2.6, Security NFR). Only public broadcast channels may be included in the Telegram Channel List.
Assumptions:	Channels in the Telegram Channel List regularly post Singapore events with a date and a location. LLM extraction is mostly accurate, and Users can verify event details through the source link.
Notes and Issues:	The initiating actor is NULL — the run is triggered by the system's own schedule, not by an external actor. A "Scheduler" actor is not introduced, as a scheduler is internal to the system. The Telegram Channel List is edited in configuration by the project team; this is a configuration task rather than system behaviour, so it has no use case.

- **Class diagram of entity classes**
- **Key boundary classes and control classes**

3.2 Build the Conceptual Model

3.2.1 From the Use Case Descriptions and the terms in the Data Dictionary, identify the initial entity, boundary and control objects and their attributes.

3.2.2 From your understanding of the classes' responsibilities, apply the stereotypes for entity, boundary or control classes.

3.2.3 In the Class Diagrams, depict these classes and the relationships between them (association, generalization, aggregation).

Class(TEMPLATE)

Name of Object: Class_name
Operations / Methods: Actions, Stop, move()
Attributes: e.g. Colour(), ID(), Size()

ENTITY CLASSES

Name of Object: Neighbourhood
Operations / Methods: getFlatWeightedMetric(), getResaleMatchPercentage(), getCoordinates()
Attributes: planningAreaName, boundaryGeometry, totalFlats, rawFactorValues

Name of Object: HDBBlock
Operations / Methods: calculateWalkTime(), countNearbyAmenities()
Attributes: postalCode, coordinates, flatCount, walkTimeToMRT, distToAmenities

Name of Object: FactorOptions
Operations / Methods: validateInput(), getWeightForFactor()
Attributes: selectedFactors, importanceWeights, flatType, budgetRange, commuteDest

Name of Object: Recommendation
Operations / Methods: calculateMarginalContribution(), generateExplanation()
Attributes: winningNeighbourhood, runnerUp, dominantFactor, marginalContribution

Name of Object: Event
Operations / Methods: isExpired(), isDuplicate(), getDistance()
Attributes: title, eventCategory, startDateTime, venueName, coordinates, sourceUrl

Name of Object: Route
Operations / Methods: getEstimatedArrival(), getSummary()
Attributes: travelMode, travelTimeMinutes, distance, summary, stepDirections

Name of Object: Carpark

Operations / Methods: isDataFresh(), getDistanceFromVenue()
Attributes: carparkID, name, coordinates, availableLots, lastUpdated

Name of Object: DataSnapshot
Operations / Methods: validateData(), isStale()
Attributes: snapshotDate, datasetVersions, isValidated

BOUNDARY CLASSES

Name of Object: WhereToLiveUI
Operations / Methods: displayNeighbourhoods(), getUserPreferences(), showResults()
Attributes: selectedNeighbourhoodList, factorInputFields, resultsDisplay

Name of Object: WhereToGoUI
Operations / Methods: getSearchCriteria(), displayEvents(), showParking()
Attributes: categoryCheckboxes, dateFilter, startLocationInput, resultsList

Name of Object: ScoreBreakdownUI
Operations / Methods: openModal(), closeModal()
Attributes: formulaDisplay, factorComparisonTable, excludedFactorsList

Name of Object: OneMapAdapter
Operations / Methods: geocodeAddress(), fetchRoute()
Attributes: apiKey, tokenExpiry, routingEndpoint

Name of Object: LTADataMallAdapter
Operations / Methods: fetchCarparkLots()
Attributes: apiKey, carparkEndpoint

Name of Object: LLMServiceAdapter
Operations / Meth ods: extractEventData()
Attributes: apiEndpoint, modelIdentifier

CONTROL CLASSES

Name of Object: ComparisonController
Operations / Methods: runComparison(), checkFactorData()

Attributes: activeSnapshot, scoringEngineReference

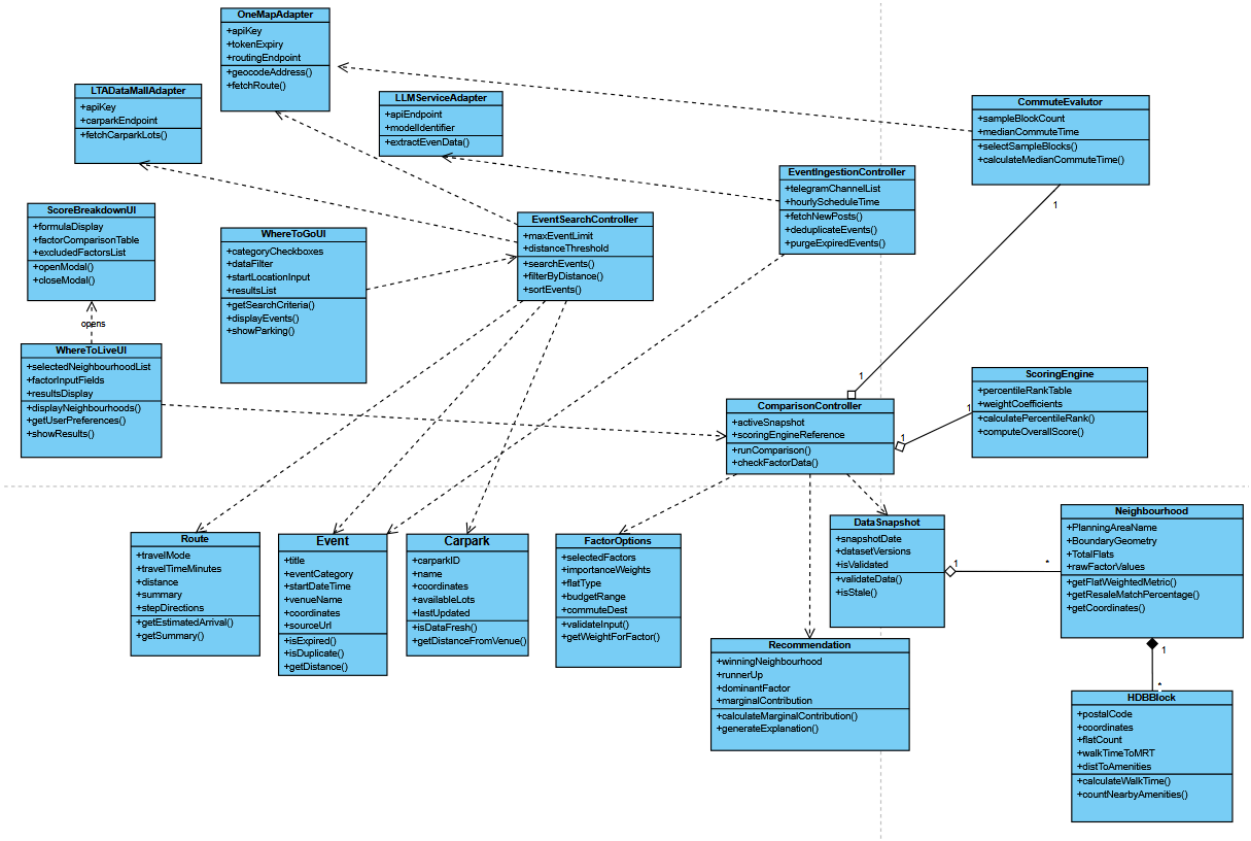
Name of Object: ScoringEngine
Operations / Methods: calculatePercentileRank(), computeOverallScore()
Attributes: percentileRankTable, weightCoefficients

Name of Object: CommuteEvaluator
Operations / Methods: selectSampleBlocks(), calculateMedianCommuteTime()
Attributes: sampleBlockCount, medianCommuteTime

Name of Object: EventSearchController
Operations / Methods: searchEvents(), filterByDistance(), sortEvents()
Attributes: maxEventLimit, distanceThreshold

Name of Object: EventIngestionController
Operations / Methods: fetchNewPosts(), deduplicateEvents(), purgeExpiredEvents()
Attributes: telegramChannelList, hourlyScheduleTime

RELATIONSHIPS

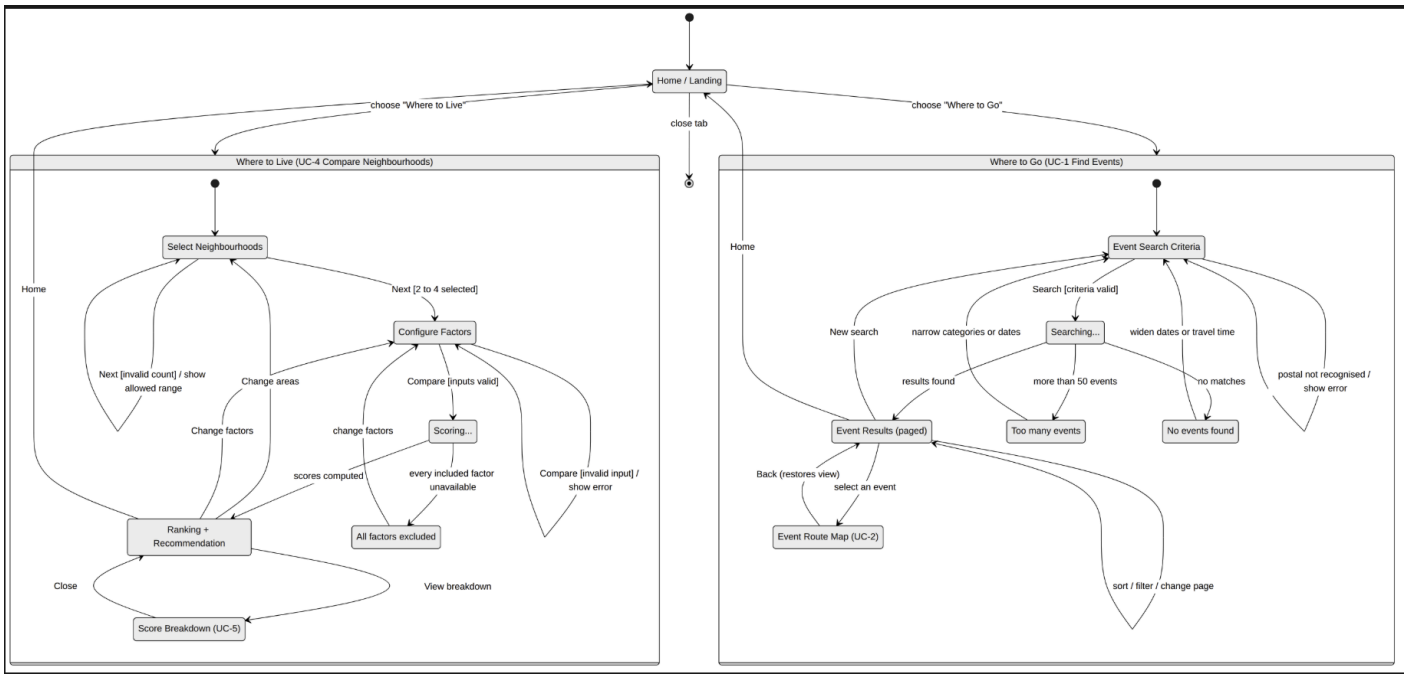


- **Initial Dialog map**

3.3 Build a Dynamic Model

3.3.1 Based on the UI mockups developed in Lab#1, model the system behavior as a State Machine of UIs. This state machine of UIs is referred to as a *Dialog Map* of the system. If this process reveals deficiencies in your UI flow, refine and further specify your user interface design. See page 420 of Fox for a detailed description of Dialog Maps.

3.3.2 Expect to move back and forth between the Conceptual Model and the Dynamic Model because studying interactions and behavior will reveal additional attributes, operations and relationships of classes.



- **Your PDF report on the AI's technology stack recommendation**

Prepare a short PDF report containing (1) a brief description of the uploaded material [full reproductions not necessary], (2) your text prompt and the original AI's chat response, (3) your written opinion from 3.4.4.

1)

The uploaded material documents the requirements and design of a neighbourhood comparison application for Singapore. The proposed system allows users to select between two and four URA Planning Areas, specify their preferences for five comparison factors such as public transport and greenery- and receive a ranked recommendation based on weight scores.

The document contains the system's functional and non-functional requirements, a Data Dictionary defining key terms, a Use Case Diagram and detailed Use Case Descriptions, and UI mockups illustrating the proposed interface. It also describes how government data from sources such as data.gov.sg, URA, LTA, NParks and MOH will be retrieved, processed, scored and periodically refreshed. The material further specifies requirements relating to usability, performance, reliability, accuracy, transparency, compatibility and security.

2)

The text prompt is "Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team".

The AI's chat response is

"For your report, I'd phrase it something like:

Technology Stack: The system will be implemented as a Python-based web application using Flask for the backend and HTML, CSS and JavaScript for the frontend. Pandas will be used for structured government data processing, while GeoPandas and Shapely will be used for geospatial processing involving URA Planning Areas and facility locations. Chart.js will be used to visualise comparison scores, and Leaflet.js will be used to display neighbourhood locations and boundaries. Government data will be periodically processed through a Python-based data refresh pipeline and stored as a local versioned data snapshot. Git and GitHub will be used for version control and team collaboration, while pytest will be used for automated testing.

In short: Python + Flask + Pandas + GeoPandas/Shapely + JavaScript + Chart.js + Leaflet + JSON/CSV + Git is probably the best balance of *easy enough for your team* and *technically credible for this project*."

3)

Our opinion is that the suggestion of Flask by ChatGPT is something we can consider seriously. Our team has experience with python and it provides a simple way to build the web application without introducing too much complexity. Since this is a course project, we want to focus on successfully implementing our features and ensuring they meet the requirements stated in our documentation.